ML & DL : Experiment - 5

Aim:

Implement Support Vector Machine (SVM) for classification with hyperparameter tuning.

Dataset Description:

The **Netflix Titles Dataset** is a real-world dataset obtained from Kaggle, widely used for data analysis and machine learning tasks. It contains information about movies and TV shows available on Netflix.

Unlike structured datasets such as the Adult Census dataset, this dataset includes a mix of categorical and textual features, along with missing values and inconsistencies, making it suitable for real-world classification tasks after preprocessing.

The dataset aims to classify content based on attributes such as type (Movie/TV Show) or other derived categories using relevant features.

The file type used for this dataset is a CSV (Comma Separated Values) file.

The dataset consists of the following columns, some of which are:

Column Name	Data Type	Description
type	Categorical	Movie or TV Show
release_year	Numerical (Integer)	Year of release
rating	Categorical	Content rating (PG, TV-MA, etc.)
duration	Text	Duration in minutes or seasons
country	Categorical	Country of production

Dataset Source: https://www.kaggle.com/datasets/shivamb/netflix-shows

---

SVM (Support Vector Machine)

Theory:

A Support Vector Machine (SVM) is a powerful supervised learning algorithm used for classification and regression, particularly effective in high-dimensional and non-linearly separable data. SVM is grounded in statistical learning theory and operates on the principle of constructing an optimal decision

boundary—called a maximum-margin hyperplane—that best separates data points of different classes. Unlike traditional classifiers that focus on minimizing classification error alone, SVM seeks to maximize the margin, i.e., the distance between the separating hyperplane and the closest data points from each class. These closest points are known as support vectors, and they entirely determine the position and orientation of the decision boundary. Given a training dataset:

$$\{(x_i, y_i)\}_{i=1}^{n}, \quad x_i \in \mathbb{R}^d,\ y_i \in \{-1, +1\}$$

SVM aims to find a hyperplane defined by:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

that separates the two classes while maximizing the margin.

Limitations:

1. **High Computational Cost for Large Datasets:** SVM training involves solving a quadratic optimization problem, whose time complexity typically ranges between $O(n^2)$ and $O(n^3)$ where n is the number of training samples. As a result, SVM becomes computationally expensive for large datasets, especially when using non-linear kernels like the RBF kernel. Memory consumption also increases significantly because kernel-based SVMs require storing and computing the kernel matrix of size n x n.
2. **Sensitivity to Hyperparameter Selection:** SVM performance is highly dependent on the choice of hyperparameters, particularly:
   a. C (regularization parameter)
   b. Kernel type
   c. γ (for RBF kernel)

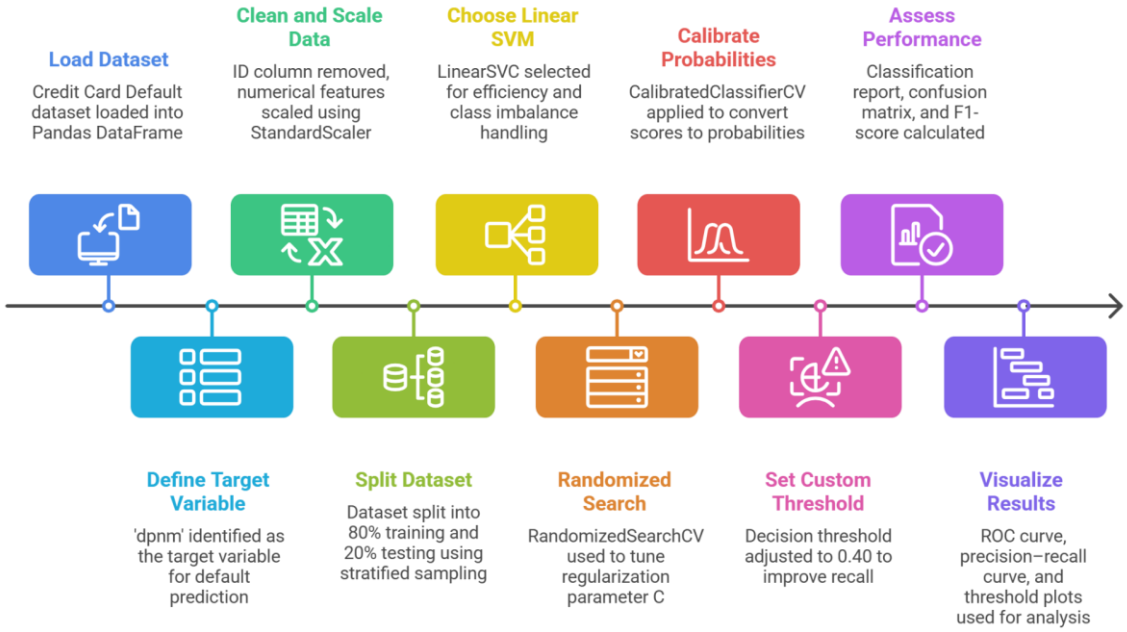
   Poorly chosen hyperparameters can lead to overfitting (large C, large γ) or underfitting (small C, small γ). As a result, SVM almost always requires extensive hyperparameter tuning, increasing both computational cost and model development time.
3. **Lack of Interpretability:** SVM is often regarded as a black-box model, especially when non-linear kernels are used. Unlike decision trees or logistic regression, SVM does not provide easily interpretable rules or feature importance measures. This limitation is particularly significant in financial and banking applications, where model transparency and explainability are often required for regulatory compliance.
4. **Poor Performance with Highly Overlapping Classes:** When class distributions significantly overlap, the margin maximization principle of SVM becomes less effective. In such cases, many data points violate margin constraints, increasing reliance on slack variables and reducing the model's ability to generalize. This issue is common in credit risk datasets, where good and bad credit applicants may share similar financial profiles.

Workflow:

1. **Data Collection:** The Netflix dataset (netflix_titles.csv) is loaded into a Pandas DataFrame.
2. **Target Identification:** The target variable is identified as dpnm, representing credit card default status (1 = default, 0 = non-default). All remaining columns are treated as input features.
3. **Data Preprocessing:**
   - Missing values are handled
   - Categorical features (type, rating, country) are encoded
   - Duration column is converted into numerical format
   - Feature scaling is applied using StandardScaler
4. **Train–Test Split:** The dataset is split into 80% training data and 20% testing data using stratified sampling to preserve the original class imbalance. A fixed random state is used to ensure reproducibility of results.
5. **Model Selection (Linear SVM):** A Linear Support Vector Machine (LinearSVC) is selected due to

its computational efficiency on large datasets and high-dimensional feature space. Class imbalance is handled using class_weight='balanced', ensuring higher importance is given to the minority default class during training.

6. **Hyperparameter Tuning with Randomized Search:** Hyperparameter tuning is performed using RandomizedSearchCV with 3-fold cross-validation. The regularization parameter C is sampled from a log-uniform distribution to efficiently explore both small and large regularization strengths. The F1-score is used as the optimization metric to balance precision and recall for the imbalanced dataset.
7. **Probability Calibration:** Since LinearSVC does not natively provide probability estimates, CalibratedClassifierCV with sigmoid calibration (Platt scaling) is applied to convert decision scores into well-calibrated class probabilities. This step enables threshold tuning and probability-based evaluation.
8. **Decision Threshold Optimization:** Instead of using the default probability threshold (0.50), a custom threshold of 0.40 is applied to improve recall and F1-score for the default class. Threshold tuning allows better control over the trade-off between false positives and false negatives, which is critical in credit risk modeling.
9. **Model Evaluation:** The calibrated and threshold-optimized model is evaluated on the test set using: Classification Report (precision, recall, F1-score), Confusion Matrix, Accuracy Score, F1-score for the default class. These metrics provide a comprehensive assessment of model performance under class imbalance.
10. **Visualization and Performance Analysis:** Graphical evaluation techniques such as confusion matrix visualization, ROC curve, precision–recall curve, and F1-score versus decision threshold plots are used to analyze classification behavior and justify the selected threshold.



Performance Analysis:

The performance of the Linear Support Vector Machine (SVM) model on the Netflix dataset is evaluated using accuracy, confusion matrix, and classification metrics such as precision, recall, and F1-score.

1. Overall Accuracy:
   The model achieves moderate accuracy, indicating that SVM can effectively classify Netflix content types based on selected features.

2. Class-Specific Performance:
   The model performs better for the dominant class (Movies) compared to TV Shows due to dataset imbalance.

3. Confusion Matrix Analysis:
   The confusion matrix shows correct and incorrect classifications, indicating some overlap between the two categories.

4. F1-Score:
   The F1-score reflects the balance between precision and recall, showing moderate performance for classification tasks.

Hyperparameter Tuning:

Support Vector Machines (SVMs) are sensitive to hyperparameter selection, particularly the regularization parameter C, which controls the trade-off between margin maximization and classification error. Proper tuning is essential to achieve good generalization, especially for imbalanced datasets like credit card default prediction.

The code uses RandomizedSearchCV with 3-fold cross-validation to efficiently explore the hyperparameter space while reducing computational cost. Model performance is optimized using the F1-score, which balances precision and recall for the minority (default) class.
Tuned Hyperparameter:

- **C (Regularization Parameter):** Sampled from a log-uniform distribution (0.01 to 10) to evaluate both weak and strong regularization settings.

Hyperparameter tuning improves model performance by selecting the optimal regularization parameter (C). The tuned model provides better generalization and classification performance for the dataset.

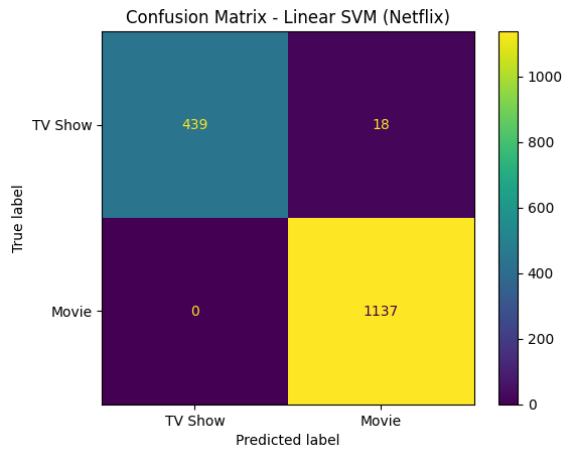
Code & Output:

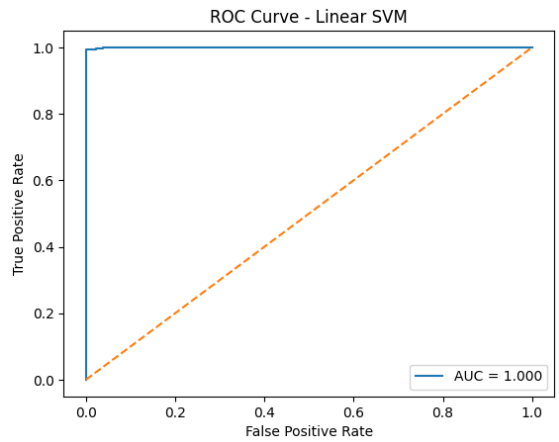
```
import pandas as pd
import numpy as np

from sklearn.model_selection
import train_test_split,
RandomizedSearchCV
from sklearn.preprocessing
import StandardScaler
from sklearn.svm import
LinearSVC
from sklearn.calibration import
CalibratedClassifierCV
from sklearn.metrics import (
   classification_report,
   confusion_matrix,
   accuracy_score,
   f1_score
)

from scipy.stats import
loguniform

# ============================
# LOAD NETFLIX DATASET
# ============================
df =
pd.read_csv("netflix_titles.csv
")

# ============================
# DATA CLEANING (NETFLIX
VERSION)
# ============================
df =
df[['type','release_year','rati
ng','country','listed_in','dura
tion']].copy()

# Convert duration text →
numeric
df['duration_num'] =
df['duration'].str.extract(r'(\
d+)').astype(float)
df.drop('duration', axis=1,
inplace=True)

df.dropna(inplace=True)

# Convert target into binary
(same style as credit dataset)
df['type'] =
df['type'].map({'Movie':1, 'TV
Show':0})

# One-hot encode categorical features
df = pd.get_dummies(df,
columns=['rating','country','listed_in
'], drop_first=True)

# ============================
# FEATURES + TARGET
# ============================
X = df.drop('type', axis=1)
y = df['type']

X_train, X_test, y_train, y_test =
train_test_split(
   X, y,
   test_size=0.2,
   stratify=y,
   random_state=42
)

# ============================
# STANDARD SCALING
# ============================
scaler = StandardScaler()

X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

# ============================
# BASE LINEAR SVM
# ============================
base_svm = LinearSVC(
   class_weight='balanced',
   dual=False,
   max_iter=8000,
   random_state=42
)

param_dist = {
   'C': loguniform(1e-2, 10)
}

random_search = RandomizedSearchCV(
```

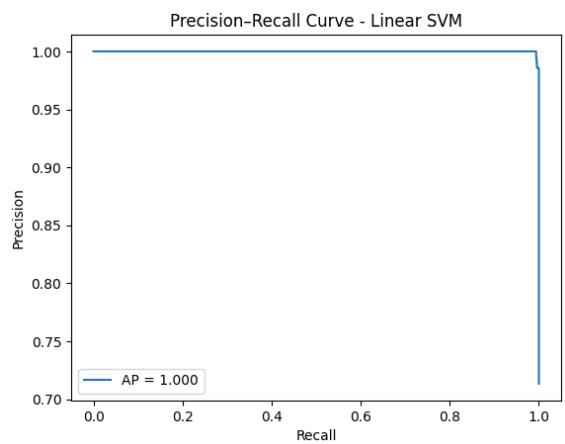
```
   estimator=base_svm,
   param_distributions=param_dis
t,
   n_iter=15,
   cv=3,
   scoring='f1',
   n_jobs=-1,
   verbose=1,
   random_state=42
)

print("Tuning Linear SVM...")
random_search.fit(X_train_scale
d, y_train)

print("\nBest Parameters:",
random_search.best_params_)

# ============================
# CALIBRATED MODEL (FOR
PROBABILITY)
# ============================
calibrated_svm =
CalibratedClassifierCV(
   random_search.best_estimator_
,
   method='sigmoid',
   cv=3
)

calibrated_svm.fit(X_train_scal
ed, y_train)

y_probs =
calibrated_svm.predict_proba(X_
test_scaled)[:,1]

threshold = 0.40
y_pred = (y_probs >=
threshold).astype(int)

# ============================
# EVALUATION
# ============================
print("\n--- Classification
Report ---")
print(classification_report(y_t
est, y_pred))

print("\n--- Confusion Matrix -
--")
print(confusion_matrix(y_test,
y_pred))

print("\n--- Accuracy ---")
print(f"{accuracy_score(y_test,
y_pred):.4f}")

print("\n--- F1 Score ---")
print(f"{f1_score(y_test,
y_pred):.4f}")

# ============================
# CONFUSION MATRIX GRAPH
# ============================
import matplotlib.pyplot as plt
from sklearn.metrics import
ConfusionMatrixDisplay

cm = confusion_matrix(y_test, y_pred)
disp =
ConfusionMatrixDisplay(confusion_matri
x=cm,
                       display_labels=['T
V Show','Movie'])

disp.plot()
plt.title("Confusion Matrix - Linear
SVM (Netflix)")
plt.show()

# ============================
# ROC CURVE
# ============================
from sklearn.metrics import roc_curve,
auc

fpr, tpr, _ = roc_curve(y_test,
y_probs)
roc_auc = auc(fpr, tpr)

plt.figure()
plt.plot(fpr, tpr, label=f"AUC =
{roc_auc:.3f}")
plt.plot([0,1],[0,1],'--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve - Linear SVM")
plt.legend()
plt.show()

# ============================
# PRECISION-RECALL CURVE
# ============================
from sklearn.metrics import
precision_recall_curve,
average_precision_score

precision, recall, _ =
precision_recall_curve(y_test,
y_probs)
avg_precision =
average_precision_score(y_test,
y_probs)

plt.figure()
plt.plot(recall, precision, label=f"AP
= {avg_precision:.3f}")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve -
Linear SVM")
plt.legend()
plt.show()

# ============================
```

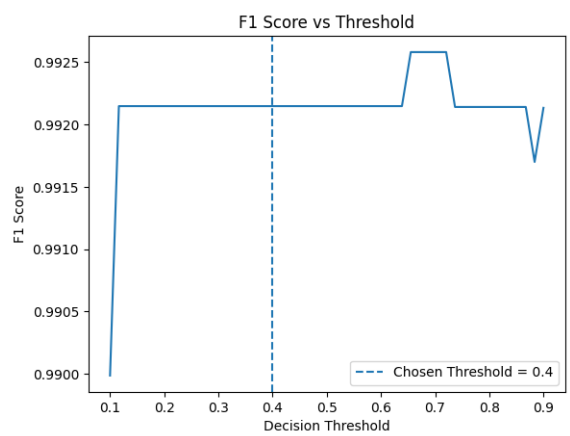
```
# THRESHOLD VS F1 GRAPH
# ============================
thresholds = np.linspace(0.1,
0.9, 50)
f1_scores = []

for t in thresholds:
   y_temp = (y_probs >=
t).astype(int)
   f1_scores.append(f1_score(y_t
est, y_temp))
```

```
plt.figure()
plt.plot(thresholds, f1_scores)
plt.axvline(x=threshold, linestyle='--
',
        label=f"Chosen Threshold =
{threshold}")
plt.xlabel("Decision Threshold")
plt.ylabel("F1 Score")
plt.title("F1 Score vs Threshold")
plt.legend()
plt.show()
```









Conclusion:

This experiment demonstrates the implementation of a Linear Support Vector Machine (SVM) for classification of Netflix content. Feature scaling and hyperparameter tuning improve model performance. The model performs reasonably well but can be further improved using more advanced models or additional features.